

LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

Section	Section 8: Limitations and Challenges
Lecture	Lecture 8.3: Technical Limitations

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Explain context window constraints and describe their practical implications for professional AI use.
- Identify the computational resource requirements of large language models and their impact on cost and latency.
- Describe the memory and reasoning limitations of current language models and explain why they matter for task design.
- Apply practical workarounds for key technical limitations without over-engineering simple workflows.
- Match AI task design to the actual technical capabilities and limitations of current models.

Introduction

Understanding the technical limitations of generative AI is as important as understanding its capabilities. Practitioners who deploy AI tools without grasping their structural constraints—context windows, computational costs, memory limitations, and reasoning weaknesses—inevitably design workflows that either hit hard limits unexpectedly or fail to mitigate known failure modes. Technical literacy about limitations enables more reliable, cost-effective, and appropriately scoped AI deployments.

The technical limitations covered in this lecture are not temporary obstacles that will disappear with the next model release. While each generation of models pushes these limits further—context windows have expanded from thousands to millions of tokens; reasoning capabilities have improved substantially—the fundamental constraints remain present in all current systems. Context windows are finite; computation is expensive; models have no persistent memory across sessions; and certain types of reasoning remain reliably difficult regardless of model size.

This guide provides practitioners with a working knowledge of the most impactful technical limitations and practical approaches for working within them. The goal is not to be discouraged by limitations but to design AI workflows that are robust to them—leveraging what models do well while mitigating where they reliably struggle.

Core Concepts Explained

Context Window Constraints

Definition: The context window is the maximum amount of text a model can process in a single interaction—both input and output combined. Early models had windows of approximately 1,000 tokens; current leading

models support 128,000 to over one million tokens, but limits still exist and vary by model and deployment configuration.

Business relevance: Context window limits affect document processing (documents longer than the window must be chunked or summarised), long conversation coherence (early conversation turns fall out of context in extended sessions), and the amount of reference material that can be provided in a prompt. Understanding these limits prevents unexpected truncation failures in production workflows.

AI Practitioner relevance: AI practitioners designing document processing pipelines must account for context window limits in their architecture. Chunking strategies, summarisation intermediate steps, and retrieval-augmented generation are the standard approaches for handling documents that exceed the effective context window.

Real-world example: Lawyers using AI to analyse lengthy litigation discovery documents—potentially thousands of pages—cannot simply feed all documents into a single prompt. Law firms like Allen & Overy use chunking and summarisation pipelines: documents are split into segments, each segment is summarised, and the summaries are then synthesised. This approach works within context limits while still enabling AI-assisted analysis of large document sets.

Risks or limitations: Extending context windows does not fully solve the 'lost in the middle' problem: research has shown that language models attend less reliably to information positioned in the middle of very long contexts than to information near the beginning or end. Practitioners using large context windows should place the most critical information at the start or end of the context, not in the middle.

Computational Cost and Latency

Definition: Large language model inference requires significant computation—proportional to model size and output length. This creates cost (commercial API pricing per token) and latency (time between prompt submission and response completion) that must be factored into workflow design.

Business relevance: For organisations deploying AI at scale—thousands of daily interactions or automated processing of large document sets—computational cost is a significant operational expense. Workflow design choices (model size, output length, frequency of generation) have direct cost implications. Latency affects user experience and determines whether AI assistance is feasible in real-time interactive applications.

AI Practitioner relevance: AI practitioners designing enterprise AI deployments should model computational costs for projected usage volumes before deployment. Decisions about which tasks use large frontier models versus smaller, cheaper local models should be driven by the quality-cost trade-off appropriate to each use case.

Real-world example: Microsoft's deployment of AI assistance across its Office 365 suite (now Microsoft Copilot) required significant infrastructure investment to manage latency at hundreds of millions of users. The company uses a tiered model strategy: lightweight models handle simple tasks (spell check, basic formatting suggestions) while frontier models are reserved for complex tasks (drafting, summarisation, multi-step analysis), managing both cost and latency at scale.

Risks or limitations: Practitioners sometimes default to using the largest available model for all tasks out of an assumption that larger equals better. For many routine tasks, smaller models produce equivalent quality at significantly lower cost and latency. Task-model matching is a meaningful cost optimisation that does not require sacrificing output quality.

Memory and Reasoning Limitations

Definition: Current language models have no persistent memory across sessions—each conversation starts fresh unless prior context is explicitly provided. Within a session, models also struggle with tasks requiring

tracking many simultaneous variables, consistent long-range reasoning, and reliable multi-step logical operations.

Business relevance: For business applications requiring continuity across sessions—personalised customer service, ongoing project support, longitudinal analysis—the absence of persistent memory requires an explicit memory architecture (storing and retrieving relevant prior context) rather than relying on the model to remember previous interactions.

AI Practitioner relevance: AI practitioners designing conversational AI systems must architect memory explicitly. Common approaches include: summarising prior conversation turns and prepending the summary to new turns, storing structured user profiles in databases and injecting relevant fields into each prompt, and using vector databases to retrieve semantically relevant prior interactions.

Real-world example: Customer service platforms like Intercom that deploy AI assistants maintain a customer history database separate from the language model. When a customer initiates a new interaction, relevant history (previous complaints, purchase history, account status) is retrieved from the database and injected into the system prompt. The model treats this as provided context rather than memory—enabling apparent continuity without persistent model memory.

Risks or limitations: Injecting large amounts of context from external memory into prompts consumes context window space and increases latency. Practitioners must balance the richness of provided context against the cost and performance implications. Summarisation of prior context is often preferable to verbatim injection.

Reasoning Reliability and Consistency

Definition: Current language models exhibit inconsistent reasoning: they may solve a problem correctly when framed one way and fail on an equivalent problem framed differently; they struggle with multi-step logical chains that exceed a certain complexity; and they can produce different answers to the same question across different conversation sessions.

Business relevance: For any business application where reasoning reliability is critical—automated decision support, structured analysis, compliance assessment—the inconsistency of model reasoning is a fundamental risk. Workflows that depend on reliable reasoning without human oversight are inappropriate for current models.

AI Practitioner relevance: AI practitioners should not architect workflows that treat language model reasoning as deterministic or reliable without verification. Chain-of-thought prompting improves reasoning visibility; human review of reasoning chains is still required for high-stakes applications.

Real-world example: OpenAI's evaluation of GPT-4 on standardised reasoning benchmarks (MMLU, GPQA, HumanEval) showed strong average performance but significant variance across problem types and phrasings. Problems that elicit the same logical reasoning pattern but frame it differently can produce correct answers on one framing and incorrect on another. This variance is why benchmark scores do not translate directly into reliability for specific deployment tasks.

Risks or limitations: Model reasoning has improved dramatically with each generation and with techniques like chain-of-thought prompting and reinforcement learning from human feedback. However, the gap between impressive benchmark performance and reliable production deployment remains real. Practitioners should test models extensively on their specific task distribution, not rely on general benchmark scores to predict performance.

Practical Applications

The following scenarios illustrate how to design AI workflows that are robust to technical limitations.

Long Document Processing Pipeline

A financial services firm analyses earnings call transcripts (often 15,000-20,000 words) that exceed the practical limit for consistent attention in a single prompt. Their pipeline: documents are split into sections (executive summary, Q&A, forward guidance), each section is summarised independently using a smaller, faster model, and then the summaries are synthesised into an analyst report by a frontier model. Critical information (key financial figures, guidance changes) is placed at the start of each section summary to counter the 'lost in the middle' problem. The pipeline processes each document in under two minutes at a fraction of the cost of single-pass frontier model processing.

Persistent Memory Customer Service

A telecommunications company deploys an AI customer service assistant that maintains continuity across customer interactions. Rather than relying on model memory, the system maintains a structured customer record in a CRM database: last three interaction summaries, current issues, account tier, and relevant product history. When a customer initiates a new interaction, this record is retrieved and injected into the system prompt (capped at 500 tokens to manage context cost). The assistant can reference prior interactions accurately despite having no intrinsic memory.

Cost-Optimised Multi-Model Deployment

A content company uses AI across multiple workflows with different quality-cost requirements. Simple categorisation tasks (tagging articles by topic) use a small, fast model at low cost per classification. Medium-complexity tasks (generating article summaries from provided text) use a mid-tier model at moderate cost. Only complex, high-value tasks (generating original long-form content from a brief) use frontier models at premium cost. This tiered approach reduces the average cost per content item by 60% compared to using a frontier model for all tasks, without measurable quality loss on categorisation and summarisation tasks.

Best Practices

- **Design for context window limits from the start.** Do not build document processing workflows that assume unlimited context. Architect chunking, summarisation, or RAG from the beginning for any use case involving documents longer than 10,000 words.
- **Place critical information at the start or end of prompts.** Research on the 'lost in the middle' phenomenon shows that language models attend most reliably to the beginning and end of long contexts. Organise prompts and context accordingly for tasks where specific information must be reliably attended to.
- **Match model size to task complexity.** Use smaller, cheaper models for routine, high-volume tasks; reserve frontier models for complex, high-value, or quality-critical tasks. This discipline meaningfully reduces cost without sacrificing quality where quality matters.
- **Architect memory explicitly for applications requiring continuity.** Do not assume the model will remember prior interactions. Design external memory stores, structured context injection, and conversation summarisation from the outset for any application requiring inter-session continuity.
- **Test reasoning reliability on your specific task distribution.** General benchmark scores do not predict performance on specific deployment tasks. Test extensively on representative samples from your actual task distribution before relying on model reasoning for consequential decisions.

Common Mistakes and Misconceptions

Misconception: Assuming large context windows eliminate chunking requirements.

Correct approach: Even with million-token context windows, the 'lost in the middle' attention problem persists. Very long contexts are not reliably processed—critical information placed in the middle of a very long context may receive less model attention than information at the start or end. Architectural solutions remain relevant even with large windows.

Misconception: Using frontier models for all tasks regardless of complexity.

Correct approach: Task-model matching—using the smallest model that produces adequate quality for a given task—is a standard cost optimisation that does not compromise quality where it matters. Frontier models should be reserved for tasks that genuinely require their capabilities.

Misconception: Designing production workflows that rely on model memory across sessions.

Correct approach: Models have no persistent memory across sessions. Workflows requiring inter-session continuity must architect explicit memory stores—database records, conversation summaries, structured context injection—rather than expecting the model to remember prior interactions.

Misconception: Trusting model reasoning for high-stakes decisions without human review.

Correct approach: Model reasoning is probabilistic and inconsistent. The same logical problem framed differently can produce different answers. Human review of reasoning chains—not just final conclusions—is required for any decision with significant consequences.

Planning Checklist: Technical Limitations Assessment for AI Deployment

Before deploying an AI workflow, answer these questions to identify which technical limitations require mitigation in your architecture.

- ☐ Context window: Will any input documents or conversation histories exceed 10,000 words? If yes, design a chunking or summarisation strategy before building.
- ☐ Lost in the middle: Does your workflow depend on the model reliably attending to information in the middle of a long context? If yes, restructure to place critical information at the start or end.
- ☐ Computational cost: Have you estimated the monthly cost at projected usage volumes? Is cost within acceptable bounds for the task's business value?
- ☐ Latency: Does your use case require real-time or near-real-time responses? If yes, test latency at realistic load before deployment.
- ☐ Memory continuity: Does your application require the model to remember prior interactions across sessions? If yes, design an external memory store and context injection mechanism.
- ☐ Reasoning reliability: Does your workflow depend on model reasoning for consequential decisions? If yes, add chain-of-thought prompting and mandatory human review of reasoning chains.
- ☐ Task-model matching: Have you considered whether a smaller model would produce adequate quality for this task at lower cost and latency?

How to use this: Address each flagged item in your deployment architecture before launch. Revisit this checklist when the task, model, or usage volume changes significantly.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

1. A legal team is building an AI workflow to analyse 500-page due diligence documents. What architectural design is most appropriate for the context window limitation?

- A. Use a model with a million-token context window and submit the full document in a single prompt.
- B. Use a chunking and summarisation pipeline: split the document into sections, summarise each, and synthesise the summaries.
- C. Ask the AI to read only the executive summary and make inferences about the full document.
- D. Use multiple separate AI models simultaneously to process different sections.

Answer: B. Use a chunking and summarisation pipeline: split the document into sections, summarise each, and synthesise the summaries.

Why: *Even with large context windows, chunking and summarisation is more reliable due to the 'lost in the middle' attention problem; it also manages cost by using smaller models for summarisation and reserving frontier models for synthesis.*

2. A customer service AI assistant is deployed to help customers who may have had prior interactions with the support team. The model has been giving contradictory advice to returning customers because it does not remember prior conversations. What is the correct fix?

- A. Switch to a larger model with better memory.
- B. Extend the context window to include all prior conversations.
- C. Design an external memory store that retrieves and injects relevant prior interaction summaries into each new session's system prompt.
- D. Instruct the model to ask customers to summarise their prior interactions.

Answer: C. Design an external memory store that retrieves and injects relevant prior interaction summaries into each new session's system prompt.

Why: *Models have no intrinsic cross-session memory; persistent continuity requires explicit architecture—a database of prior interactions retrieved and injected as context, not a model capability.*

3. A company is using a frontier AI model for all tasks, including simple document categorisation at high volume. What is the most impactful optimisation?

- A. Fine-tune the frontier model on categorisation examples to make it faster.
- B. Reduce the number of categories to decrease model reasoning load.
- C. Switch categorisation to a smaller, cheaper model adequate for the task, reserving the frontier model for complex tasks.
- D. Increase the context window allocation to improve categorisation accuracy.

Answer: C. Switch categorisation to a smaller, cheaper model adequate for the task, reserving the frontier model for complex tasks.

Why: *Task-model matching is a standard cost optimisation: simple, high-volume tasks like document categorisation typically do not require frontier model capabilities; smaller models produce equivalent quality at substantially lower cost.*

Reflection Questions

1. Review your current AI workflows. Which ones involve documents or conversations that might exceed context window limits? What chunking or summarisation strategies would address this?
2. Have you experienced unexpected costs from AI tool usage? In retrospect, were you using the most expensive model where a smaller one would have sufficed?
3. Think of an AI application in your organisation that requires continuity across sessions. Is external memory explicitly architected, or is the system relying implicitly on model memory that does not exist?
4. Where in your current AI workflows is model reasoning used without human review? How consequential are the decisions being supported by that reasoning?
5. How would you explain the 'lost in the middle' problem to a colleague who believes that larger context windows have solved document processing limitations?

Key Takeaways

- Context window limits affect document processing, long conversation coherence, and the amount of reference material in prompts; chunking and summarisation pipelines are the standard mitigation.
- The 'lost in the middle' problem means that critical information placed in the middle of very long contexts receives less reliable attention; place key information at the start or end of prompts.
- Computational cost and latency scale with model size and output length; task-model matching—using the smallest model adequate for a task—is an important cost optimisation.
- Language models have no persistent memory across sessions; applications requiring continuity must explicitly architect external memory stores and context injection.
- Model reasoning is probabilistic and inconsistent; reasoning reliability improves with chain-of-thought prompting but human review remains necessary for high-stakes applications.
- Technical limitations are not temporary obstacles but inherent properties of current model architectures; robust AI deployment design plans for them from the outset.
- Test AI systems on your specific task distribution—not just general benchmarks—before relying on them for consequential decisions in production.

Recommended Next Actions

6. Complete the technical limitations assessment checklist for one current AI deployment. Identify which limitations are unmitigated and design a fix for the highest-risk one.
7. Estimate the monthly cost of your highest-volume AI workflow at current usage. Assess whether task-model matching could reduce this cost without quality loss.
8. Review one application that requires continuity across sessions. Document how memory is currently handled and identify whether an explicit memory architecture is needed.
9. Design a chunking and summarisation pipeline for a long document processing task in your domain. Specify the chunking strategy, summarisation model, and synthesis approach.